Advanced OOP Concepts in Python: Bank Account System

Contents

1 Introduction

In previous OOP lessons, we learned:

- class
- object
- methods
- constructor

Now we go one level deeper:

Real-world class design

We will build a:

Bank Account System

2 Class Attributes vs Instance Attributes

```
class BankAccount:
    currency_options = [...]
    bank_name = "XYZ Bank Pvt. Ltd."
```

These are:

Class attributes

Meaning:

- shared by all objects
- same for every account

3 Instance Attributes

```
self.owner_name = owner_name
self.__balance = balance
```

These belong to:

individual object

Each account has its own balance.

4 Data Validation in Constructor

```
if not isinstance(balance, int):
    raise Exception(...)
```

```
if balance < 0:
    raise Exception(...)
```

This ensures:

- only valid data enters system

5 Private Variable Concept

```
self.__balance
```

Double underscore means:

Private variable

It is not directly accessible outside class.

6 Protected vs Private

- `_balance` → protected (convention)
- `__balance` → private (name mangling)

Python does NOT fully enforce privacy, but suggests it.

7 Name Mangling Concept

```
a1._BankAccount__balance
```

Python internally renames private variables like this.
But:

We should NOT access it directly

8 Encapsulation

Encapsulation means:

Bundling data + methods + protecting data

Example:

- balance is protected
- methods control access

9 Property Decorator

```
@property
def balance(self):
    return self.__balance
```

Now we can access:

```
a1.balance
```

like a variable, but actually it's a method.

10 Why Property?

It gives:

- controlled access
- security
- clean syntax

11 Setter Method

```
@balance.setter
def balance(self, new_value):
```

This allows:

- updating value safely

12 Validation in Setter

```
if not isinstance(new_value, int):
    raise Exception(...)
```

```
if new_value < 0:
    raise Exception(...)
```

So even updates are protected.

13 Currency System Concept

```
currency_options = [
    ("NPR", "Rs.", 1),
    ("USD", "$", 0.006),
    ("INR", "Re.", 1/1.6),
]
```

Each tuple contains:

- currency code
- symbol
- conversion rate

14 Instance Currency Selection

```
self.display_currency = BankAccount.currency_options[0]
```

Each user can choose display currency.

15 Method: Currency Display

```
def get_currency_display(self):
    return self.display_currency[1]
```

Returns symbol like:

- Rs.
-

16 Method: Converted Balance

```
def get_balance_display(self):
    return self.__balance * self.display_currency[2]
```

This converts base currency (NPR) into selected currency.

17 Method: Show Account Details

```
def show_account_details(self):
    print(f"""
        Bank Name    : {self.bank_name}
        Account Name: {self.owner_name}
        Balance      : {self.get_currency_display()} {self.
           get_balance_display()}
    """)
```

This is:

Abstraction in action

18 Abstraction

Abstraction means:

Hiding internal logic, showing simple interface

User only sees:

- account details

Not internal calculations.

19 Real-world Usage Pattern

In real systems:

- balance is never directly modified
- only methods control it

This prevents:

- invalid data
- security issues

20 OOP Pillars Summary

20.1 Encapsulation

- bundling data + methods
- protecting data

20.2 Abstraction

- hiding complexity
- showing simple interface

20.3 Inheritance

- reuse code from parent class

20.4 Polymorphism

- same function, different behavior

Example:

```
len("text") -> 4
len([1,2,3]) -> 3
```

Same function, different outputs.

21 Important Insight

We are slowly moving from:

writing code

to:

designing systems

22 Final Realization

Everything in this system:

- class
- object
- method
- attribute

is part of a bigger idea:

> Object Oriented Programming is about modeling real world logically